

RL Вопросы и ответы

Базовые понятия RL

Есть среда, которая описывается состояниями s_i и переходами (действиями, $A(s_i)$) между ними, и агент совершая последовательные действия в ней получает награду $R(s_i, a_i)$. Всё взаимодействие со средой описывается как марковский процесс (MDP), который полагает, что вероятность перейти в следующее состояние среды $p(s')$ зависит только от текущего состояния и выбранного действия. Если обозначить траекторию $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ из T шагов и политику π (функцию выбора действий/вероятность выбрать действие в данном состоянии), то вероятность траектории будет выражена как

$$P(\tau|\pi) = p_0(s_0) \prod_{t=0}^{T-1} p_i(s_{t+1}|s_t, a_t) \pi(a_t|s_t)$$

И можно выразить матожидание для политики как

$$J(\pi) = \int_{\tau} R(\tau|\pi) R(\tau) = \mathbb{E}_{\tau \sim \pi}[R(\tau)]$$

Задача обучения заключается в поиске политики π^* , которая максимизирует доход.

$$\pi^* = \arg \max_{\pi} J(\pi)$$

Обычно задача не ограничивается конечным числом шагов, поэтому рассматривается

функция накопленного дохода $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ с параметром дисконтирования $0 \leq$

$\gamma < 1$, который нужен, чтобы доход был конечным (для сравнимости) и мы давали приоритет награде в текущий момент.

Метод кросс-энтропии (табличный + нейронная сеть)

Метод кросс-энтропии относится к эволюционным алгоритмам и не требует дифференцируемости среды или явного вычисления функции ценности состояний. Суть заключается в итеративном обновлении параметров стратегии таким образом, чтобы разницу между текущим распределением действий и неким целевым распределением, которое формируется исключительно из наиболее успешных траекторий. На каждой итерации агент генерирует N независимых эпизодов τ , сортирует их по итоговому доходу G_t и отсекает топ- q процентов лучших, формируя «элитную» выборку. Модель обновляет параметры распределения политики так, чтобы они отдавали предпочтение элитному распределению.

В табличном случае (для дискретных и небольших пространств) новая стратегия $\pi(a|s) = A_{s,a}$ вычисляется как эмпирическое распределение: частота выбора конкретного действия a в состоянии s среди всех элитных эпизодов.

$$\pi(a|s) = \frac{\sum_{s_t, a_t \in Elites} [s_t = s][a_t = a]}{\sum_{s_t, a_t \in Elites} [s_t = s]}$$

В случае использования нейронной сети (веса W) в качестве аппроксиматора стратегии задача сводится к классическому обучению с учителем: сеть обучается предсказывать действия из элитной выборки на основе соответствующих состояний, минимизируя функцию потерь кросс-энтропии (фактически метод максимального правдоподобия для политики).

$$\pi^* = \arg \max_{\pi} \sum_{s_i, a_i \in Elite} \log \pi(a_i | s_i)$$

$$W_{i+1} = W_i + \alpha \nabla \left[\sum_{s_i, a_i \in Elite} \log \pi_{W_i}(a_i | s_i) \right]$$

Этот метод вычислительно прост, но обладает низкой эффективностью использования данных и применим преимущественно в задачах с конечными эпизодами. Отличается от генетических алгоритмов тем, что не использует мутацию и скрещивание и быстрее сужает область поиска.

V(s), Q(s, a) - что это такое?

Функция ценности состояния $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$ отражает математическое ожидание суммарного дисконтированного дохода G_t , если агент начинает взаимодействие со средой из состояния s и строго следует фиксированной политике π . Она позволяет оценить глобальную перспективность нахождения в конкретном состоянии с учетом всех будущих шагов, абстрагируясь от мгновенной награды. Функция $V(s)$ не отвечает на вопрос, какое действие следует выбрать в данный момент, если агенту неизвестна динамика среды (вероятности переходов P).

Для принятия решений без знания модели используется функция ценности действия $Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$. Она показывает ожидаемый доход при условии, что в состоянии s агент совершает конкретное действие a , а все последующие шаги генерируются согласно политике π . Связь между ними может быть описана рекурсивными уравнениями Беллмана: $V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s, a)$. Знание оптимальной функции

$Q^*(s, a)$ позволяет определить оптимальную политику π^* , выбирая $\arg \max_a Q^*(s, a)$ в каждом состоянии, не используя модель переходов среды.

Метод Policy iteration

Policy iteration — это классический алгоритм динамического программирования для поиска абсолютно оптимальной политики, который требует полного знания Марковского процесса принятия решений (включая матрицу вероятностей переходов и функцию наград). Алгоритм базируется на строгом чередовании двух фаз: оценки политики и улучшения политики. Оценка состояний:

$$V_{(i+1)}(s) = \max_a \sum_{s'} P(s'|s, a) \cdot [r(s, a, s') + \gamma V_i(s')] = \max_a Q_i(s, a)$$

Алгоритм:

Инициализируем $V^\pi(s) = 0$; $\pi(s) = a$ - произвольное действие.

Пока π изменяется, $\pi(s) \leftarrow \arg \max_{a \in A(s)} Q^\pi(s, a)$

На фазе оценки решается система линейных уравнений Беллмана для вычисления истинной функции $V^\pi(s)$ для текущей политики π . На практике это делается итеративным применением оператора ожидаемого дохода ко всем состояниям до полной математической сходимости.

Получив точные значения $V(s)$, алгоритм запускает фазу улучшения: для каждого состояния формируется новая, жадная политика. Она детерминированно выбирает действие, максимизирующее сумму мгновенной награды и дисконтированной ценности следующего состояния. Согласно теореме об улучшении политики, новая стратегия математически гарантированно будет генерировать доход больше или равный доходу предыдущей стратегии. Процесс оценки и улучшения повторяется до тех пор, пока политика не перестанет изменяться, что означает достижение глобального оптимума π^* .

Q-learning, SARSA

Q-learning и SARSA (State-Action-Reward-State-Action) — это табличные алгоритмы обучения на основе временных разностей. Обновление текущих оценок $Q(s, a)$ происходит опираясь на собственные предсказания для следующих состояний среды $Q(s', a')$, не дожидаясь фактического завершения эпизода. Далее α обозначает learning rate (скорость обучения).

Q-learning является off-policy алгоритмом. При обновлении он берет максимум по всем возможным действиям в следующем состоянии: $Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$.

Это означает, что он обучается оптимальной функции Q^* независимо от того, какие случайные шаги делает агент в реальности.

SARSA является on-policy алгоритмом. Его формула обновления использует значение Q того действия a' , которое агент выбрал на следующем шаге: $Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma Q(s', a') - Q(s, a)]$.

Из-за этого SARSA выучивает более консервативную политику, так как она учитывает риски и негативные последствия от собственных исследовательских действий в процессе обучения.

Expected SARSA - вместо функции значения для следующего действия использовать мат. ожидание от всевозможных действий (стабильнее, но нужно много замеров).

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \mathbb{E}[Q(s', a')|s'] - Q(s, a)]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \sum_a \pi(a|s') Q(s', a) - Q(s, a)]$$

Exploration vs Exploitation

Исследование против Использования — проблема компромисса в обучении с подкреплением, вызванная необходимостью балансировать между максимизацией уже известной выгоды и сбором новой информации. Чрезмерная эксплуатация приводит к тому, что агент застревает в первом же найденном локальном оптимуме. Чрезмерное исследование не позволяет агенту накопить полезный опыт и конвертировать его в награду. Классическим решением является ϵ -greedy стратегия: с вероятностью $1 - \epsilon$ агент действует жадно (эксплуатирует), а с вероятностью ϵ выбирает случайное действие (исследует). На практике параметр ϵ плавно снижается от 1.0 почти до нуля - набраться опыта и закрепить успех.

DQN

Deep Q-Network — Q-learning с использованием глубоких нейронных сетей для работы в средах с непрерывным или высокоразмерным пространством состояний (где табличный метод применить невозможно). Нейросеть (с весами θ) аппроксимирует значения $Q(s, a; \theta)$ для всего множества возможных дискретных действий.

Прямое применение нейросетей в алгоритмах обучения на основе временных разностей приводит к катастрофической нестабильности и расходимости алгоритма из-за высокой временной корреляции последовательных наблюдений и нестационарности распределения целевых меток. Для решения этих проблем применяются Experience Replay и Target Network.

Experience replay

Experience Replay - циклический буфер фиксированного размера, сохраняющий прошлые взаимодействия. Для обучения мы сэмплируем случайные выборки из буфера, что защищает от временной корреляции данных и делает распределение данных более репрезентативным. Переиспользование данных в целом повышает эффективность.

Target network

Проблема Q-learning, что мы обновляем оценки сравнивая предсказание сети с целевым значением, которое тоже зависит от следующего состояния. Веса нейросети постоянно меняются, что приводит к нестабильности и расходящейся динамике. Поэтому вводится целевая сеть с весами (θ^-), которая является замороженной копией основной нейросети. Веса в ней обновляются каждые условно тысячу шагов, либо через экспоненциальное усреднение.

Обновление весов нейросети происходит по правилу: $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))$

Double DQN

В стандартном Q-learning выбор $\max_{a'} Q(s', a')$ на начальном этапе очень зашумлён, поэтому чтобы нейросеть сама себя не дестабилизировала задачи выбора оптимального действия и оценку ценности разносят на разные нейросети (замороженная target network с весами θ^-).

Активная выбирает действие: $a^* = \arg \max_a Q(s', a; \theta)$

Замороженная оценивает: $Y = R + \gamma Q(s', a^*; \theta^-)$

Policy gradient

Вместо нейросети для оценки $Q(s, a)$ используем нейросеть для предсказания вероятностного распределения действий (политики $\pi_\theta(a|s)$). Стараемся максимизировать мат. ожидание суммарного дохода.

$$\nabla J = \mathbb{E}_{s \sim d(s), a \sim \pi(a|obs(s))} \nabla \log \pi(a|s) Q(s, a)$$

Главным образом отличается от Q-learning тем, что не нужно моделировать каждое состояние (что проблемно для непрерывных или многомерных пространств) и считать для него $\arg \max_a Q(s', a; \theta)$ путём перебора всех действий ($Q(s, a)$ вычислить тяжелее, чем выбрать само действие). По определению Q-learning - жадный алгоритм, а policy gradient - вероятностный, что позволяет выучивать смешанные стратегии. Также алгоритм стабильнее, чем DQN, поскольку использует градиентный подъём по целевой функции (плавнее временных разностей, но и измерения нужно накопить).

REINFORCE

Policy gradient алгоритм использующий метод Монте-Карло для оценки мат. ожидания суммарного дохода (требуется меньше измерений). Оценка Монте-Карло обладает нулевым математическим смещением, но высокой дисперсией. Что приводит к медленной и нестабильной сходимости в задачах со сложным пространством состояний.

z - сессия текущей политики, сэмплируем N штук. Правильнее нормировать, вычитая из $Q(s, a)$ некий baseline ($b(s)$), потому что разные участки траектории могут иметь сильно разные значения и заглушать вклад остальных.

$$\nabla J = \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in z_i} \nabla \log \pi_\theta(a|s) (Q(s, a) - b(s))$$

Actor-critic

Для policy-based алгоритма было бы хорошо взять в $b(s) = V(s)$, но нейросеть моделирует только политику - нужно обучать Критика.

Рассмотрим функцию преимущества $A(s, a) = r + \gamma V(s') - V(s)$

$$\nabla J = \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in z_i} \nabla \log \pi_\theta(a|s) A(s, a)$$

Это быстрее поскольку можно зафиксировать $A(s, a) = \text{const}$ и вычислять градиенты не имея всей траектории.

Применение RNN для генерации последовательностей

Рекуррентные нейронные сети генерируют последовательность (токенов) поэлементно, храня информацию о предыстории в скрытом состоянии h .

$$h_{i+1} = \sigma(W_{hh}h_i + W_{xh}x_i + b_h)$$

W_{hh} - часть нейросети, отвечающая за извлечение полезных признаков из истории.

W_{xh} - часть нейросети, отвечающая за извлечение полезных признаков из текущего обрабатываемого токена x_t .

σ - функция активации ($\tanh$, sigmoid).

Чтобы превратить h_i в токен на выход, надстраивают внешний линейный слой - например, умножают на матрицу W_{hx} , получают вектор логитов и применяют Softmax, получая вероятности токенов. Начало обозначается специальным токеном BOS, а конец - EOS. Это нужно, чтобы модель чётко понимала границы на этапе обучения - где контекста нет и где закончить инференс.

Для обучения обычно выбирают последовательность фиксированной длины: на каждом шаге вычисляют loss и затем суммируют или усредняют. На длинных участках появляется проблема затухания градиента. Её можно решать через skip-connections (вход суммируется с выходом слоя) или более продвинутые LSTM (но лучше transformer).

Применение RL к дообучению моделей генерации последовательностей

Допустим мы обучили модель при помощи supervised learning

- она точно повторяет тексты и поэтому у неё плохо с фантазией;
- она повторяет даже те тексты, что мы бы не хотели;
- она обучилась на одной сфере, но мы хотим применить в другой.

Её можно улучшить через RLHF (Reinforcement Learning from Human Feedback): генерируем несколько вариантов и даём человеку оценить. Так мы можем обучить модель Критика. Проблемно, в RL модель может найти лазейки и застрять в нежелательном состоянии.

Можно использовать PPO (Proximal Policy Optimization) модификацию Actor-Critic. Изменение политики удерживаем в рамках $[1 - \epsilon, 1 + \epsilon]$ ($\epsilon \approx 0.1$), чтобы не было резких

скачков Loss. Т.е. мы обрубаем величину $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$